Heaven's Light is our guide

Rajshahi University of Engineering & Technology

Department of Electrical & Computer Engineering

Course Name: Digital Techniques

Course Code: ECE-2112

Experiment No: 01

Experiment Name: Introduction to Logisim Evolution

Submission Date: 26/07/2026

Submitted By:

Sharmista Saha

Roll no: 2410013

Reg no: 1066

Submitted To:

Mst. Mazeda Noor Tasnim

Lecturer,

Department of ECE, RUET

1 Task No. 1: Installation of Logisim

1.1 Theory

Logisim-Evolution is an advanced, open-source educational tool engineered specifically for the design, simulation, and analysis of digital logic circuits. It offers a user-friendly graphical interface where students can construct circuits visually using standard schematic symbols and analyze real-time signal propagation. Key features include hierarchical circuit design supporting modular sub-circuits, built-in oscilloscope views for timing diagram analysis, combinatorial analysis tools for automated truth table and Boolean expression generation, and board-level integration capabilities. This makes it an indispensable environment for mastering digital logic foundations prior to physical hardware prototyping on FPGA or breadboard platforms.

1.2 Screenshots (Step by Step)

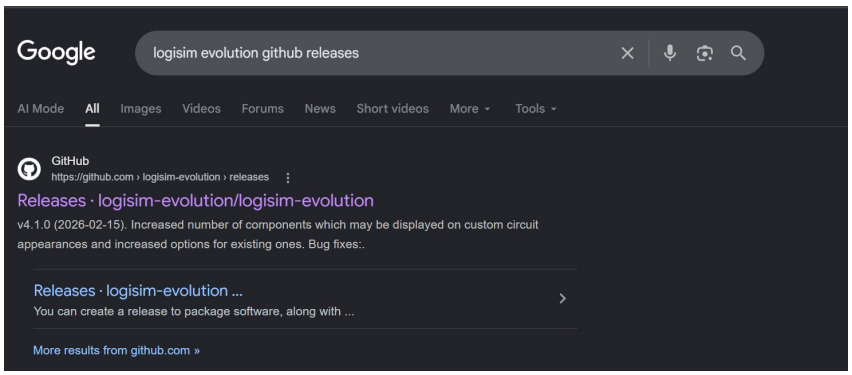


Figure 1: Step 1: Going to Logisim Evolution Github release

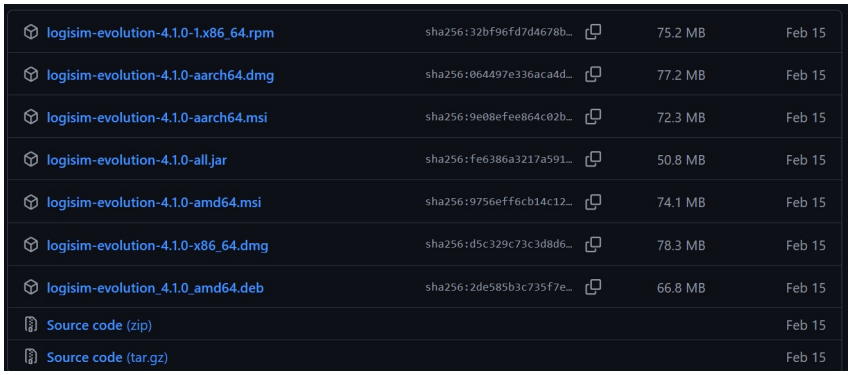


Figure 2: Step 2: Choosing msi file for Windows 11

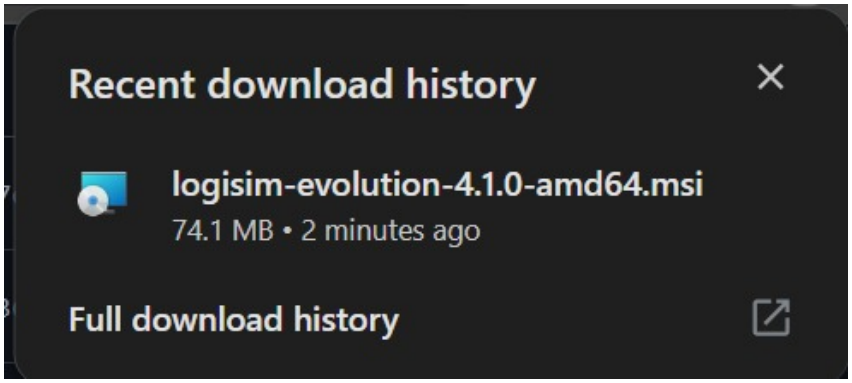


Figure 3: Step 3: Completing Downloading the .msi file

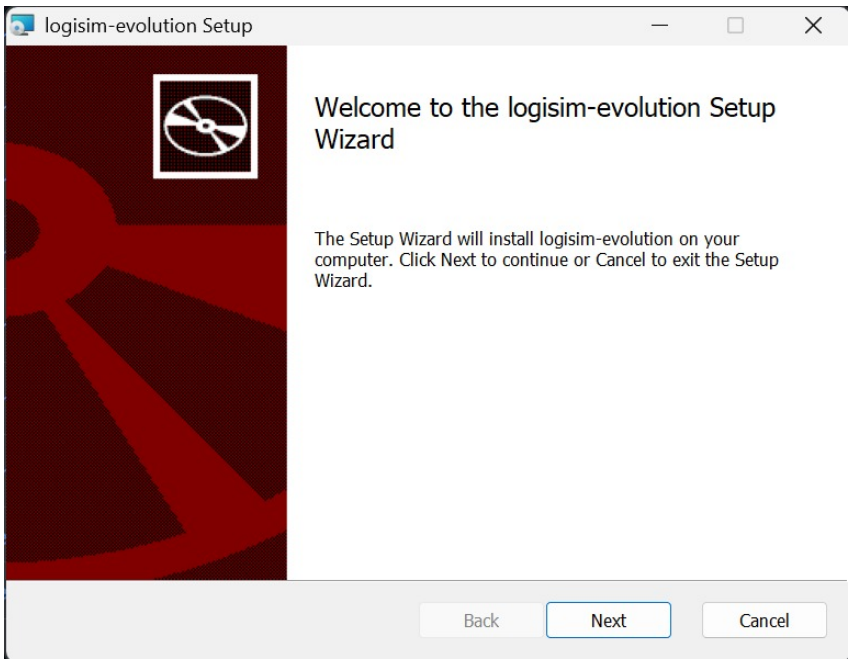


Figure 4: Step 4: Starting Initialization Process

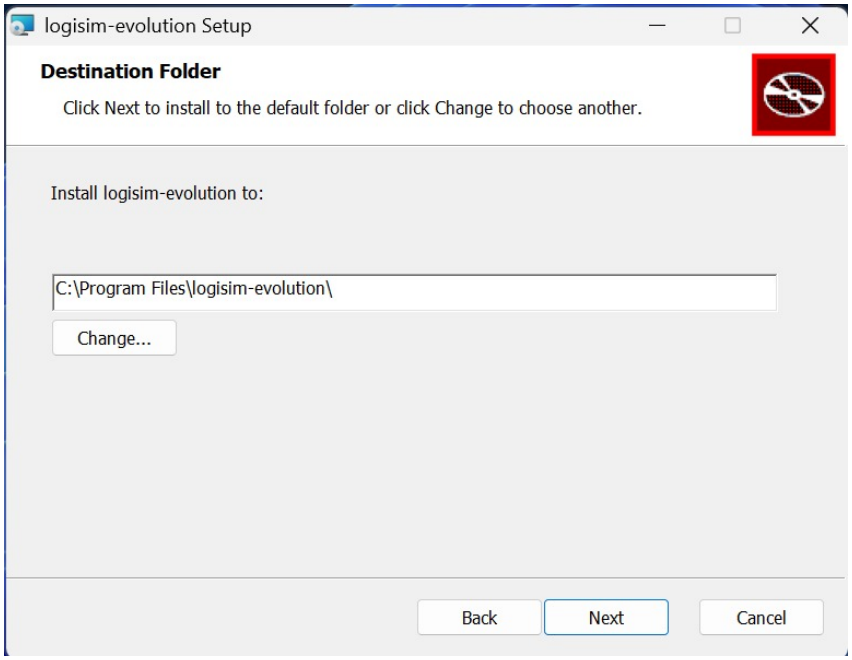


Figure 5: Step 5: Selecting Destination folder for Logisim Evolution

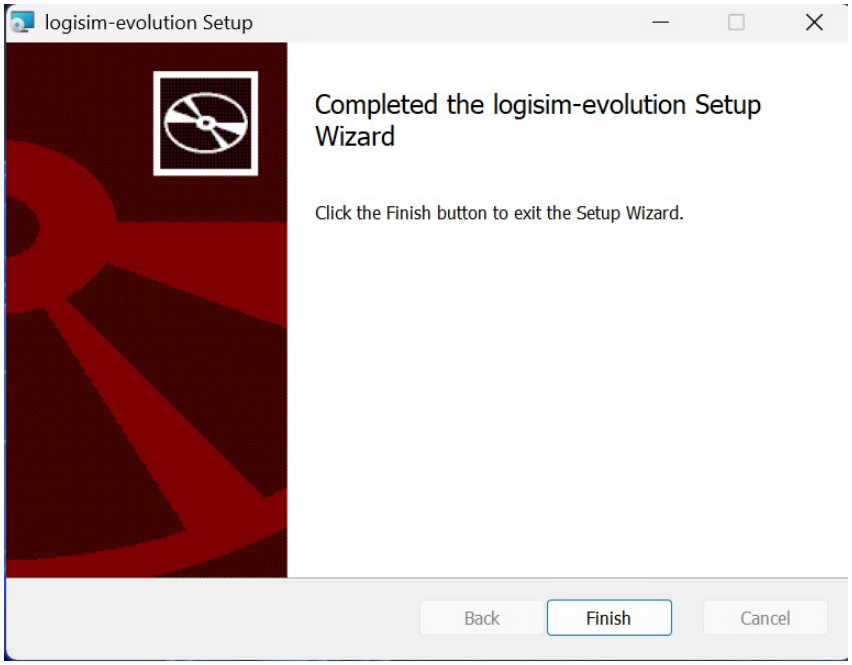


Figure 6: Step 6: Completing Full Installation Process

1.3 Discussion

In this task, Logisim-Evolution was successfully installed on the Windows system by executing the standard installation package and configuring the runtime environment. Navigating through the installation steps ensured that all necessary dependencies, such as Java runtime integrations, were correctly set up. Exploring the primary workspace subsequently familiarized

us with the component explorer toolbar, the main drawing canvas, and the attribute table panel. Understanding how to manage project properties and utilize these foundational interface tools is vital for efficiently constructing, routing, and troubleshooting complex combinational and sequential digital systems in the subsequent tasks of this laboratory course.

2 Task No. 2: AND by NAND

2.1 Theory

A NAND (Not-AND) gate is universally recognized as a universal logic gate because any Boolean function or basic logic gate can be implemented entirely using combinations of NAND gates. To realize an equivalent AND operation utilizing exclusively NAND structures, the fundamental logic principle relies on double inversion. By feeding the output of a primary NAND gate into a secondary NAND gate whose inputs are tied together to act as an inverter, the net logical operation complies with Boolean algebra: $\overline{\overline{A \cdot B}} = A \cdot B$. This transformation eliminates the need for physical AND IC packages, proving useful for hardware optimization and minimizing chip counts in integrated circuit design.

2.2 Circuit Diagram

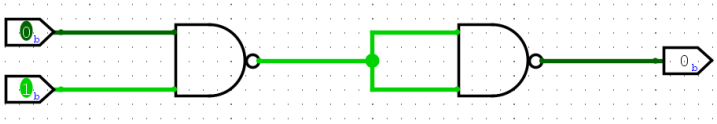


Figure 7: AND gate implemented using NAND gates

2.3 Truth Table

Input A	Input B	Output Y
0	0	0
0	1	0
1	0	0
1	1	1

Table 1: Truth table for AND by NAND

2.4 Discussion

The simulation results obtained from Logisim-Evolution strictly verify that the constructed circuit accurately mimics the behavior of a standard two-input AND gate using solely universal NAND gates. As observed in the simulation, the output terminal remains at a LOW state for all input combinations except when both input A and input B are driven HIGH, yielding a HIGH output. This performance matches the theoretical truth table precisely, successfully validating the practical application of De Morgan's laws and universal gate substitution in digital circuit design.

3 Task No. 3: OR by NAND

3.1 Theory

Implementing an OR operation using only NAND gates requires the systematic application of De Morgan's laws. According to Boolean identities, an OR function can be expressed in terms of NAND logic by individually complementing each input prior to applying a final NAND operation, yielding the expression $\overline{\overline{A} \cdot \overline{B}} = A + B$. In practice, this circuit topology utilizes two preliminary NAND gates configured as inverters (to invert inputs A and B respectively) feeding into a third terminating NAND gate. This configuration highlights how complex routing strategies can leverage universal components to achieve required logical outputs.

3.2 Circuit Diagram

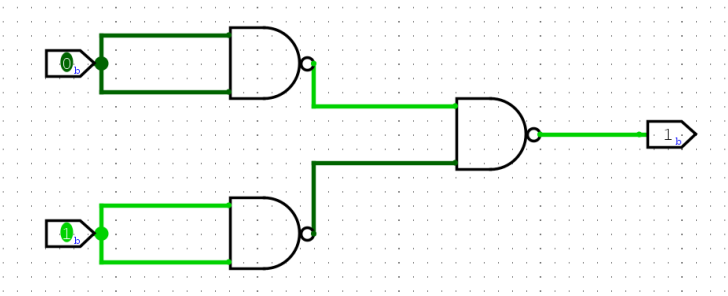


Figure 8: OR gate implemented using NAND gates

3.3 Truth Table

Input A	Input B	Output Y
0	0	0
0	1	1
1	0	1
1	1	1

Table 2: Truth table for OR by NAND

3.4 Discussion

The simulation of the NAND-based OR circuit confirmed that logical disjunction can be successfully achieved without relying on dedicated OR gates. The output responded correctly by transitioning to a HIGH state whenever either input, or both inputs simultaneously, were set to HIGH, remaining LOW only when both inputs were grounded. Minor propagation delays typical of multi-level gate structures were observable during signal transitions, reinforcing our understanding of real-world circuit timing constraints and gate-level layering effects.

4 Task No. 4: NOT by NAND

4.1 Theory

A NOT gate, commonly referred to as an inverter, performs logical negation, turning a binary HIGH input into a LOW output and vice versa. A universal NAND gate can easily be transformed into a functional inverter by short-circuiting (tying together) its input terminals to share a single common input signal A. Under this condition, the general two-input NAND expression $\overline{A \cdot B}$ collapses to $\overline{A \cdot A}$, which simplifies via idempotent laws to $\overline{A}$. This simple yet powerful configuration is widely used in digital design to generate complement signals without requiring discrete inverter chips.

4.2 Circuit Diagram

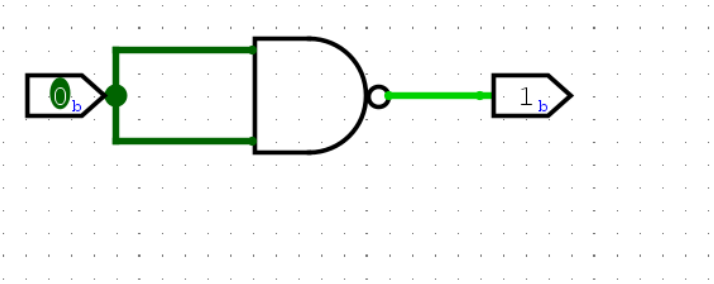


Figure 9: NOT gate implemented using a NAND gate

4.3 Truth Table

Input A	Output Y
0	1
1	0

Table 3: Truth table for NOT by NAND

4.4 Discussion

The simulation results conclusively verified that tying the input terminals of a NAND gate creates a dependable and efficient logic inverter. When input A was driven LOW (0), the output

registered HIGH (1), and conversely, driving input A HIGH (0) resulted in an immediate LOW output state. This behavior matches the expected theoretical operation, demonstrating that universal components can seamlessly perform basic inversion tasks with minimal hardware overhead.

5 Task No. 5: AND by NOR

5.1 Theory

Similar to NAND gates, NOR gates are classified as universal building blocks capable of generating any digital logic function independently. To synthesize an AND gate using strictly NOR logic, algebraic transformations based on duality and De Morgan's laws are applied. By routing each incoming signal through a separate NOR-configured inverter before combining them into a final NOR gate, the circuit executes the logical equivalence $\overline{\overline{A} + \overline{B}} = A \cdot B$. This exercise demonstrates alternative pathways for circuit minimization and highlights the flexibility inherent in universal gate families.

5.2 Circuit Diagram

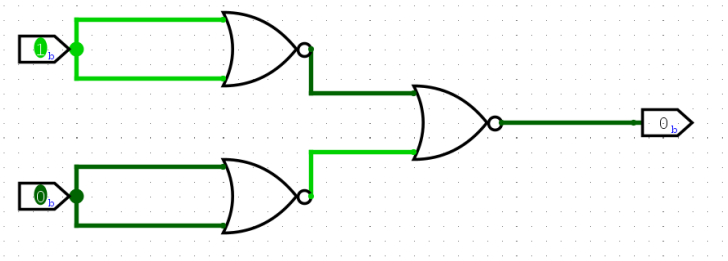


Figure 10: AND gate implemented using NOR gates

5.3 Truth Table

Input A	Input B	Output Y
0	0	0
0	1	0
1	0	0
1	1	1

Table 4: Truth table for AND by NOR

5.4 Discussion

The simulation verified that interconnecting multiple NOR gates in this specific topological arrangement successfully yields an exact AND gate response. The output indicator light remained off across all initial test states and only turned on when both inputs were simultaneously activated. This confirms the validity of the NOR-based AND design and deepens our understanding of inter-family logic conversions.

6 Task No. 6: OR by NOR

6.1 Theory

A standard OR gate can be constructed using NOR gates by cascading a primary NOR gate with an auxiliary inverter (NOT gate). Since a base NOR gate inherently performs a negated disjunction operation ($\overline{A + B}$), appending a subsequent inverter effectively neutralizes the outer negation. This restores the operational signal to the standard positive OR function: $\overline{\overline{A + B}} = A + B$. This technique illustrates how multi-stage configurations allow universal components to replicate standard logic blocks.

6.2 Circuit Diagram

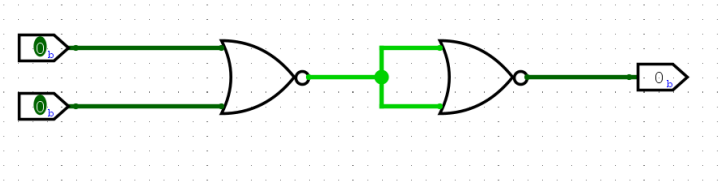


Figure 11: OR gate implemented using NOR gates

6.3 Truth Table

Input A	Input B	Output Y
0	0	0
0	1	1
1	0	1
1	1	1

Table 5: Truth table for OR by NOR

6.4 Discussion

The simulation output successfully demonstrated the functionality of the NOR-based OR circuit. The inclusion of the output inversion stage successfully corrected the inverted polarity of the base NOR gate, matching the expected truth table. Observing the signal states verified that the circuit effectively performs logical addition using only NOR structures.

7 Task No. 7: NOT by NOR

7.1 Theory

In a manner analogous to NAND-based inversion, a universal NOR gate can be configured to act as a NOT gate (inverter) by joining all input pins together to accept a single common line A. Under this structural constraint, the standard NOR logical operation $\overline{A + B}$ reduces to $\overline{A + A}$, which simplifies through Boolean idempotent laws to $\overline{A}$. This configuration provides an alternative method for generating signal complements using standard inventory parts.

7.2 Circuit Diagram

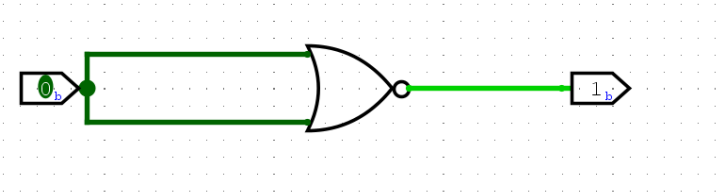


Figure 12: NOT gate implemented using a NOR gate

7.3 Truth Table

Input A	Output Y
0	1
1	0

Table 6: Truth table for NOT by NOR

7.4 Discussion

Testing the simulation model confirmed that tying together the inputs of a NOR gate yields a clean, reliable signal inversion. The output state consistently mirrored the inverse of the input value across all trials. This experiment successfully validated the theoretical transformation and showcased the versatility of universal gate architectures.

8 Task No. 8: Full Adder by Logic Circuit

8.1 Theory

A full adder is a fundamental combinational arithmetic circuit designed to compute the arithmetic sum of three single-bit binary inputs: augend A, addend B, and carry input C_{in}. Because it accounts for a carry-in bit from previous lower-order stages, it can be cascaded to add multi-bit binary numbers. The circuit generates two distinct outputs: Sum and Carry Out (C_{out}).

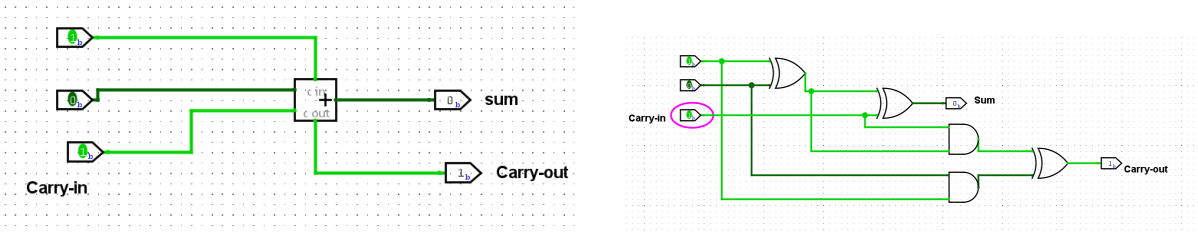
Mathematically, the logical relationships are defined by the canonical Boolean expressions:

$$\text{Sum} = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \cdot B) + (B \cdot C_{in}) + (C_{in} \cdot A)$$

Practically, a full adder can be built using two half-adders combined with an OR gate, or implemented directly via structural gate-level interconnections.

8.2 Circuit Diagram



(a) Full Adder (b) Logic implementation

Figure 13: Full Adder circuit using basic logic gates

8.3 Truth Table

Input A	Input B	Carry In (C_{in})	Sum	Carry Out (C_{out})
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 7: Truth table for Full Adder

8.4 Discussion

The simulation of the full adder circuit successfully demonstrated multi-bit arithmetic processing capabilities at the single-bit level. By cycling through all eight possible input permutations of A, B, and C_{in}, we verified that the generated Sum and Carry Out flags matched theoretical calculations perfectly. Minor transient propagation delays were visible during simultaneous input changes, highlighting the impact of gate-level depth on arithmetic circuit speed. Overall, the design proved robust and functioned precisely as required for binary addition architectures.

9 Task No. 9: Binary to BCD with 7 Segment Display

9.1 Theory

A binary-to-BCD (Binary-Coded Decimal) conversion and display system bridges internal machine-level binary data with human-readable decimal readouts. Because digital computers process information in binary formats while humans interpret data in decimal systems, conversion logic is necessary to translate binary values into individual decimal digits (Tens and Units). These separated digits are then mapped through a 7-segment display decoder to drive the corresponding segments (a through g) of a physical display. This conversion is an essential aspect of digital instrumentation, measuring equipment, and user interface design where clear numerical feedback is required.

9.2 Circuit Diagram

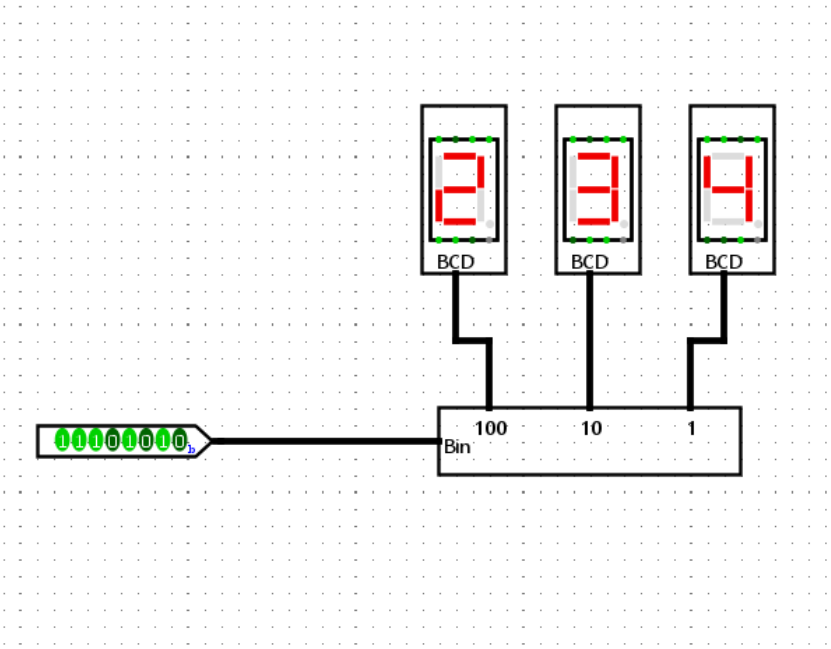


Figure 14: Binary to BCD conversion with 7-segment display circuit

9.3 Discussion

The binary-to-BCD simulation successfully interfaced raw input values with the 7-segment display subsystem. The circuit correctly decoded incoming binary states and routed the appropriate segment signals to illuminate the display accurately across various test values. Testing showed that the decoding logic maintains stability and correct numerical representation without visual flickering or decoding errors. This experiment demonstrated how complex subsystem integration can be achieved in Logisim-Evolution to design practical, human-readable digital output interfaces.